

机器学习上机实验11：神经网络-1

实验题目：PyTorch自动求导与神经网络训练

一、实验目的

1. 掌握PyTorch计算图（Computational Graph）的构建过程；
2. 理解自动求导机制（Autograd）的工作原理；
3. 观察梯度的计算、传播与累积过程；
4. 理解参数更新机制及梯度下降原理；
5. 使用PyTorch实现神经网络训练；
6. 分析训练过程中损失函数和梯度的变化规律。

二、实验环境

- Python 3.10+
- PyTorch 2.x
- NumPy
- Matplotlib

三、实验内容

实验一：PyTorch计算图与自动求导

任务1：构建计算图并计算梯度

给定如下表达式：

$$y = (wx + b)^2$$

其中：

代码块

```
1 import torch
2
3 w = torch.tensor(2.0, requires_grad=True)
```

```
4 x = torch.tensor(3.0)
5 b = torch.tensor(1.0, requires_grad=True)
```

要求

(1) 补充代码完成表达式计算：

代码块

```
1 y = _____
```

(2) 调用自动求导：

代码块

```
1 y.backward()
```

(3) 输出梯度：

代码块

```
1 print("w.grad =", w.grad)
2 print("b.grad =", b.grad)
```

思考题

1. 为什么变量 `w` 和 `b` 具有梯度？
2. 为什么变量 `x` 没有梯度？
3. `requires_grad=True` 的作用是什么？

输出计算图信息：

代码块

```
1 print(y.grad_fn)
```

继续构造：

代码块

```
1 z = w * x + b
```

输出：

代码块

```
1 print(z.grad_fn)
```

要求

记录输出结果，并说明：

- `grad_fn` 表示什么？
- 计算图中的节点是如何连接的？

任务2：损失函数梯度传播与链式法则

根据如下运算：

$$z = wx + b$$

$$a = \sigma(z)$$

$$L = a^2$$

要求

补充代码：

代码块

```
1 z = _____  
2  
3 a = _____  
4  
5 loss = _____
```

执行：

代码块

```
1 loss.backward()
```

输出：

```
代码块
1 print(w.grad)
2 print(b.grad)
```

思考题

请画出该计算图，并说明梯度是如何从 Loss 反向传播到参数 (w) 和 (b) 的。

执行：

```
代码块
1 loss.backward()
2 loss.backward()
```

观察：

```
代码块
1 print(w.grad)
```

然后执行：

```
代码块
1 w.grad.zero_()
2 b.grad.zero_()
```

再次观察梯度变化。

思考题

1. 为什么连续调用两次 `backward()` 后梯度变大？
2. 为什么训练过程中需要执行：

```
代码块
1 optimizer.zero_grad()
```

实验二：神经网络训练与梯度分析

任务1：构建XOR数据集

建立如下数据：

x1	x2	y
0	0	0
0	1	1
1	0	1
1	1	0

代码块

```
1  X = torch.tensor([
2      [0.,0.],
3      [0.,1.],
4      [1.,0.],
5      [1.,1.]
6  ])
7
8  Y = torch.tensor([
9      [0.],
10     [1.],
11     [1.],
12     [0.]
13  ])
```

任务2：构建神经网络

建立如下结构：

代码块

```
1  Input(2)
2      ↓
3  Linear(2→4)
4      ↓
5  Sigmoid
6      ↓
7  Linear(4→1)
8      ↓
9  Sigmoid
10     ↓
11  Output(1)
```

要求

补充以下代码：

代码块

```
1  import torch.nn as nn
2
3  class XORNet(nn.Module):
4
5      def __init__(self):
6
7          super().__init__()
8
9          ...
10
11     def forward(self,x):
12
13         ...
14
15     return output
```

任务3：统计网络参数

输出网络参数：

代码块

```
1  for name,param in model.named_parameters():
2
3      print(name)
4      print(param.shape)
```

思考题

计算该网络共有多少个可训练参数。

任务4：观察梯度

定义：

代码块

```
1  criterion = nn.MSELoss()
2
3  optimizer = torch.optim.SGD(
4      model.parameters(),
5      lr=0.5
```

```
6    )
```

执行一次前向传播与反向传播：

代码块

```
1    output = model(X)
2
3    loss = criterion(output,Y)
4
5    loss.backward()
```

输出：

代码块

```
1    print(model.fc1.weight.grad)
2
3    print(model.fc2.weight.grad)
```

思考题

1. 梯度表示什么含义？
2. 梯度值越大意味着什么？

任务5：观察参数更新

保存更新前参数：

代码块

```
1    old_weight = model.fc1.weight.data.clone()
```

执行：

代码块

```
1    optimizer.step()
```

输出：

代码块

```
1 print(old_weight)
2
3 print(model.fc1.weight.data)
```

思考题

1. 参数是否发生变化？
2. 参数更新的依据是什么？

任务6：完成网络训练

训练网络（训练时不要忘记使用 `optimizer.zero_grad()`）：

代码块

```
1 epochs = 2000
```

要求：

- 记录每轮训练损失；
- 绘制损失函数变化曲线。

输出

打印最终预测结果：

代码块

```
1 print(model(X))
```

观察模型是否正确学习XOR映射关系。

任务7：分析梯度变化

训练过程中记录：

代码块

```
1 model.fc1.weight.grad.norm()
2
3 model.fc2.weight.grad.norm()
```

保存各轮梯度范数。

绘制：Loss-Epoch曲线 与 Gradient Norm-Epoch曲线

思考题

1. 梯度范数为什么会随训练变化？
2. 输入层附近梯度与输出层附近梯度是否存在差异？

四、 作业要求

- **代码：**整理出代码文件，规范添加注释。
- **报告：**依据本实验内容，形成实验报告。